

SAVING COMPUTATION WITH RELEVANCE CLASSIFIERS IN LONG CONTEXT FILTERING

JOHN COLLINS (PHD), UNIVERSITY OF MANNHEIM

ABSTRACT. Extremely long context handling is a common application for Large Language Models (LLMs). Researchers have developed numerous techniques for managing information that must be filtered from among long passages of irrelevant text. However, all such techniques benefit from a mechanism that filters out truly irrelevant text before applying the LLM. Besides Retrieval-Augmented Generation, which relies on semantic similarity between the query and supporting text, the main alternative is to parse the text with another LLM agent that comprehends the task to detect and retain relevant text while discarding irrelevant passages. However, for such an agent to perform these reasoning tasks effectively, it typically must be a fairly large—and therefore computationally expensive—model. This method stub proposes an alternative process in which this agent effectively labels a subset of data, which is then used to train a smaller LLM agent with a classification head to serve as a first-pass classifier. By tuning the classification thresholds to filter out only those text passages confidently labeled as irrelevant, the smaller agent reduces the number of chunks that need to be processed by the larger agent, yielding computational savings under the right conditions. This method stub formalizes the conditions for such savings and provides a demonstration using Phi-2-Medium as the smaller agent and Phi-3 as the larger agent. The demonstration yielded a 13.30% reduction in average FLOPs per token.

Introduction

Large context handling is a significant challenge in Large Language Model (LLM) applications, often resolved at the cost of considerable computation and, consequently, energy consumption (Maliakel, Ilager, and Brandic 2025). Many tasks require fulfilling user instructions based on information dispersed across a large corpus filled with irrelevant sections. Currently, commercially available LLMs have limited context windows and reduced reasoning performance as context size increases (Schnabel et al. 2025; Wu et al. 2024). To address this, researchers have developed techniques to filter out irrelevant parts of a corpus, such as Retrieval Augmented Generation (RAG) (Lewis et al. 2021), or to recursively summarize large text segments into smaller ones that fit within a target context window (Sarathi et al. 2024). Other approaches include using memory tokens that retain important information as the LLM processes the corpus in sequential chunks (Bulatov, Kuratov, and Burtsev 2022), as well as techniques that support extremely large context windows (Ding et al. 2024). All of these methods benefit from reducing irrelevant information, either by improving task performance or by reducing the amount of text to process, thereby lowering energy consumption.

However, filtering irrelevant information introduces two risks: (1) the filtering process may consume more computation than it saves, and (2) relevant information may be inadvertently discarded.

This method stub presents a workflow aimed at mitigating both risks, within a scenario where the user has already decided to filter irrelevant text chunks from a long corpus before passing the filtered data to a downstream LLM for instruction fulfillment. The goal is to reduce overall computation, which, in principle, corresponds to reduced energy consumption (Maliakel, Ilager, and Brandic 2025). This method does not aim to improve classification accuracy or task-specific performance. Instead, it addresses a specific use case as follows.

The proposed technique is an enhancement over the baseline workflow depicted in Figure 1. In this context, the user seeks to fulfill an instruction using information from a large corpus with an LLM-system such as any of the above mentioned techniques. Let Z be this large-context handling system. Before using Z , the user aims to reduce irrelevant text from the corpus. The corpus is first split into chunks using any arbitrary splitting method. Then, another LLM (denoted as D) performs binary relevant/irrelevant classification on each chunk. The resulting boolean mask is used to filter out irrelevant text, and the retained chunks are passed to Z .

The proposed improvement, depicted in 1, involves introducing a first-pass filter using a smaller LLM with a trained classification head (denoted as C). This model outputs probabilities for relevance classification, allowing chunks confidently deemed irrelevant to be filtered out early. Since C is significantly smaller than D , and fewer chunks need to be processed by D , this yields computation savings exponentially proportional to the number of tokens filtered. However, while C is less resource-intensive than D , it still requires training and inference computation. The effectiveness of this approach depends on how many chunks can be confidently filtered out in the first pass, while keeping the risk of discarding relevant chunks minimal.

The next section formalizes this approach and presents a demonstration using real-world values from an example use case involving Phi-2 Medium as C .

Formalizing Computation Savings

Using the framework presented in Kaplan et al. (2020), we can estimate the number of Floating Point Operations (FLOPs) per token for a given context length. Having calculated the FLOPs per token for C and D , we can then proceed to estimate savings in FLOPs as follows.

Let:

- C_f : Flops per token of C
- D_f : Flops per token of D
- Θ : Proportion of tokens filtered out by C at 100% precision (meaning no relevant chunks are filtered out).

Then the compute cost per token of the baseline approach is simply D_f , and therefore the saved computation cost by the proposed alternative (ΔF) in Figure 1 is the difference between the computations needed in the baseline versus in the proposed technique, formalized as:

$$(1) \quad \Delta F = D_f - D_f \cdot (1 - \Theta) + C_f$$

Worked Example

The following briefly describes the setup used in the demonstration. Complete implementation details are available in the replication code (see 'Replication'). In this demonstration, we use question-answering tasks from Babilong Task Three (Kuratov et al. 2024), which requires answering questions based on multiple relevant passages embedded within a larger set of irrelevant text. We work with 8,000-token-long contexts, split into sequential chunks of 250 tokens each. Each chunk is augmented with an instruction prompt that includes few-shot examples and the specific question.

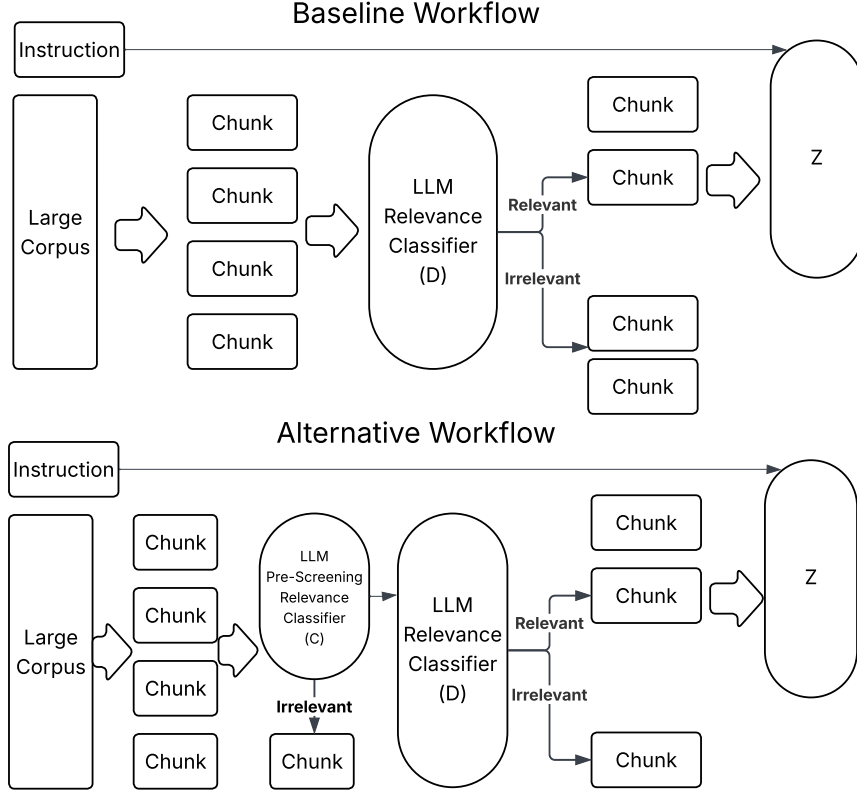


FIGURE 1. Comparison of two possible workflows for filtering out irrelevant chunks from a large corpus to fulfill an instruction.

Next, we label each chunk using an instance of Phi-2 Medium (Hughes 2023) and perform a 70-30 train-test split. To train the classifier C, we instantiate a separate Phi-2 Medium model, truncate it after the 28th transformer layer, followed by a classifier head.), and feed the outputs from this layer into a feedforward neural network.

Using this setup, we can now estimate the hypothetical computation savings if these values were applied to pre-filter irrelevant chunks before passing them to a downstream classifier, specifically, using Phi-3 (Bilenko 2024) as model D to make final relevance determinations.

Firstly, we calculate C_f using the framework from Kaplan et al. (2020) but adapted for inference. At inference, the exact average number of FLOPs per output token is impossible to estimate precisely because it depends entirely on the case at hand which corresponds to certain parameters never activating in a manner we cannot generalize. Therefore, for simplicity, we estimate the maximum FLOPs per token as if all parameters were activated exactly once and every calculation if one FLOP. This is an underestimation of the maximum amount of FLOPs, which makes C and D fairly comparable while under-estimating the presented portion of computational savings.

$$(2) \quad N \cdot n_{\text{ctx}^C}$$

Where n_{ctx^C} is the context length in tokens and N is the total number of parameters in the LLM.

Phi-2-Medium normally has 2.78 Billion parameters, but we shall only use the embedding layer, the first 28 non-embedding layers and then we append four feed-forward layers of sizes 1024, 512, 256, 128 and then a single binary classification head (sums to 1,921 parameters).

The overall number of parameters in C (N_C) is then:

- $d_{\text{embed}^C} = 131,072,000$ (embedding parameters)
- $d_{\text{ff}^C} = 78,699,520$ (feedforward parameters per layer)
- $n_{\text{layer}^C} = 28$ (number of feedforward layers)
- $d_{\text{dense}^C} = 1,921$ (number of dense layer parameters)

$$\begin{aligned}
 (3) \quad N_C &= d_{\text{embed}^C} + (n_{\text{layer}^C} \cdot d_{\text{ff}^C} + d_{\text{dense}^C}) \\
 (4) \quad &= 131,072,000 + (28 \cdot 78,699,520) + 1,921 \\
 (5) \quad &= \boxed{2,334,660,481 \text{ parameters}}
 \end{aligned}$$

And so the approximate computational cost per chunk of 250 tokens (for example) would be:

$$\begin{aligned}
 (6) \quad C_f &= N_C \cdot n_{\text{ctx}^C} \\
 (7) \quad &= 2,334,660,481 \cdot 250 \\
 (8) \quad &= \boxed{583,665,120,250 \text{ average FLOPs per token}}
 \end{aligned}$$

In our demonstration, our classifier C yielded classification probabilities for each case from which we can derive a threshold such that we only classify a case as negative if it is over 97.9% likely so and maintain no false negatives. From this we achieve a 29.95% true negative rate with minimal (1 out of 13) false negatives on the holdout test data. Therefore:

$$\boxed{\Theta = 0.2995}$$

Next, we derive the FLOPs per token for D . We use Phi-3-Medium for D , which has 14.02 Billion parameters. Unlike C , we assume there is no need to substantially alter the basic architecture of D , so we can simply estimate the computation cost of D for each chunk of 250 tokens as:

$$\begin{aligned}
 (9) \quad D_f &= N_D \cdot n_{\text{ctx}^D} \\
 (10) \quad &= 1.402e10 \cdot 250 \\
 (11) \quad &= \boxed{3.505e12 \text{ average FLOPs per token}}
 \end{aligned}$$

Finally, we can calculate the savings in FLOPs per token with an approximate average chunked context size of $n_{\text{ctx}} = 250$:

$$\begin{aligned}
\Delta F &= D_f - D_f \cdot (1 - \Theta) + C_f \\
&= 3.505e12 - 3.505e12 \cdot (1 - 0.2995) + 583,665,120,250 \\
&= \boxed{0.4660824e12 \text{ average FLOPs per token}}
\end{aligned}$$

Conclusion

By filtering out approximately 29.95% of irrelevant text chunks using Phi-2 Medium (up to layer 28) with a classifier head at 100% validation precision, we avoid expensive downstream processing by Phi-3 for a subset of tokens. However, the test precision was below 100%, having misclassified 1 out of 13 relevant chunks. Therefore, the downstream effect on query answering needs to be evaluated as a subsequent step.

This leads to an expected compute saving of:

$$\boxed{0.4660824e12 \text{ FLOPs per token on average}}$$

This corresponds to a **13.30% reduction** in compute relative to using Phi-3 directly on all tokens.

Replication

Demonstration code available openly at:

https://osf.io/aq8gu/?view_only=dc8e135532404cb1a5ecebcbf45358cc1

The code was run on Google Colab with an A100 Processor runtime.

References

- Bilenko, Misha (Apr. 23, 2024). *Introducing Phi-3: Redefining what’s possible with SLMs*. Microsoft Azure Blog. URL: <https://azure.microsoft.com/en-us/blog/introducing-phi-3-redefining-whats-possible-with-slms/> (visited on 07/31/2025).
- Bulatov, Aydar, Yuri Kuratov, and Mikhail S. Burtsev (Dec. 8, 2022). *Recurrent Memory Transformer*. DOI: 10.48550/arXiv.2207.06881. arXiv: 2207.06881[cs]. URL: <http://arxiv.org/abs/2207.06881> (visited on 07/30/2025).
- Ding, Yiran et al. (Feb. 21, 2024). *LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens*. DOI: 10.48550/arXiv.2402.13753. arXiv: 2402.13753[cs]. URL: <http://arxiv.org/abs/2402.13753> (visited on 07/30/2025).
- Hughes, Alyssa (Dec. 12, 2023). *Phi-2: The surprising power of small language models*. Microsoft Research. URL: <https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/> (visited on 07/30/2025).
- Kaplan, Jared et al. (Jan. 23, 2020). *Scaling Laws for Neural Language Models*. DOI: 10.48550/arXiv.2001.08361. arXiv: 2001.08361[cs]. URL: <http://arxiv.org/abs/2001.08361> (visited on 07/30/2025).
- Kuratov, Yuri et al. (Nov. 6, 2024). *BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack*. DOI: 10.48550/arXiv.2406.10149. arXiv: 2406.10149[cs]. URL: <http://arxiv.org/abs/2406.10149> (visited on 07/30/2025).
- Lewis, Patrick et al. (Apr. 12, 2021). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. DOI: 10.48550/arXiv.2005.11401. arXiv: 2005.11401[cs]. URL: <http://arxiv.org/abs/2005.11401> (visited on 07/30/2025).

- Maliakel, Paul Joe, Shashikant Ilager, and Ivona Brandic (June 2, 2025). *Investigating Energy Efficiency and Performance Trade-offs in LLM Inference Across Tasks and DVFS Settings*. DOI: 10.48550/arXiv.2501.08219. arXiv: 2501.08219[cs]. URL: <http://arxiv.org/abs/2501.08219> (visited on 07/30/2025).
- Sarathi, Parth et al. (Jan. 31, 2024). *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval*. DOI: 10.48550/arXiv.2401.18059. arXiv: 2401.18059[cs]. URL: <http://arxiv.org/abs/2401.18059> (visited on 07/30/2025).
- Schnabel, Tobias et al. (July 15, 2025). *Lost in Transmission: When and Why LLMs Fail to Reason Globally*. DOI: 10.48550/arXiv.2505.08140. arXiv: 2505.08140[cs]. URL: <http://arxiv.org/abs/2505.08140> (visited on 07/30/2025).
- Wu, Siye et al. (Sept. 12, 2024). *How Easily do Irrelevant Inputs Skew the Responses of Large Language Models?* DOI: 10.48550/arXiv.2404.03302. arXiv: 2404.03302[cs]. URL: <http://arxiv.org/abs/2404.03302> (visited on 07/30/2025).

A5, 68161, MANNHEIM, GERMANY

Email address: john.collins@students.uni-mannheim.de